AegisX – AI-Powered Cybersecurity Assessment Platform

System Design Document (SDD)

1. System Overview

AegisX is a modular, high-throughput, and light-footprint cybersecurity assessment platform engineered in Python. It provides centralized visibility into an application's external attack surface by executing asynchronous port probing, protocol header auditing, SSL/TLS certificate chain verification, and WHOIS/DNS intelligence gathering.

The architecture is designed around clear separation of concerns, decoupling network socket operations, threat heuristics scoring, database operations, reporting pipelines, and user interface rendering into clean, independent abstractions.

2. High-Level Architecture

The platform operates on a reactive single-tenant model, receiving user-defined target definitions through an interactive UI layer. Scanning workflows execute through asynchronous background workers managed by orchestrators, writing transient state results to an in-memory or file-backed SQLite database, before piping sanitized outputs to visualization and report generation engines.

Code snippet
```
graph TD
    User([User / Browser]) <--> UI[Streamlit UI Layer]
    UI <--> Engine[Core Scan Orchestrator]

    subgraph Scanning Modules
        Engine --> PortScan[Port Scanner & Banner Grabber]
        Engine --> HeaderAudit[HTTP Header Analyzer]
        Engine --> SSLAudit[SSL/TLS Inspector]
        Engine --> Recon[DNS & WHOIS Recon]
    end

    Scanning Modules --> RiskEngine[Risk Scoring Engine]
    RiskEngine --> Storage[(SQLite Database)]
    RiskEngine --> Export[Report Generator - ReportLab/CSV]

    subgraph Future Extensions
        Engine -.-> CVE[NVD CVE Engine]
        Engine -.-> AI[AI Explanations Engine]
    end

    Export --> User
```

3. Layered Architecture

AegisX follows a strict 4-Tier Architectural Model to ensure maintainability, testability, and portability:

1. **Presentation Layer (Streamlit):** Responsible for accepting sanitized target strings, rendering responsive progress state, generating Plotly visual indicators, and streaming dynamic results.
2. **Orchestration Layer:** Controls scan lifecycle, execution flow, parallelism thread pools, and data passing between individual core scanning modules.
3. **Domain Engine Layer (Core Logic):** Houses business rules, network socket operations, protocol analysis algorithms, risk heuristics calculations, and report rendering logic.
4. **Data & Persistence Layer:** Encapsulates SQLite schema models, configuration stores, log writers, and file system export abstractions.

4. Component Diagram

The internal functional architecture relies on isolated component interfaces connected via Python native data types (pydantic or dataclasses schemas).

Code snippet
componentStyle
component [Streamlit App Interface] as UI
component [Target Validator & Sanitizer] as Sanitizer
component [Scanner Manager Orchestrator] as Orchestrator

package "Scanning Engines" {
  component [Async TCP Scanner] as TCP
  component [HTTP/HTTPS Inspector] as HTTP
  component [TLS Certificate Parser] as TLS
  component [DNS / WHOIS Resolver] as DNS
}

component [Heuristic Scoring Engine] as Scorer
component [PDF / CSV Builder] as Builder
component [SQLite Data Access Object] as DAO

UI --> Sanitizer
Sanitizer --> Orchestrator
Orchestrator --> TCP
Orchestrator --> HTTP
Orchestrator --> TLS
Orchestrator --> DNS

TCP --> Scorer
HTTP --> Scorer
TLS --> Scorer

DNS --> Scorer

Scorer --> DAO
Scorer --> Builder
Builder --> UI

5. Module Description

Module Name	Responsibilities	Key Dependencies
ui/	Renders dashboard views, user inputs, score gauges, tables, and export download triggers.	streamlit, plotly
core/validator.py	Input sanitization, domain name extraction, and IPv4/IPv6 regex verification.	re, ipaddress
core/port_scanner.py	Asynchronous port connectivity checks, banners capture, and default port mapping.	asyncio, socket
core/header_analyzer.py	Execution of HTTP HEAD/GET probes to evaluate standard OWASP security response headers.	requests, urllib3
core/ssl_inspector.py	Deep SSL handshake analysis, authority verification, cipher suite evaluation, and expiry alerts.	ssl, cryptography
core/recon.py	Automated lookup of NS, MX, TXT, A, AAAA records and WHOIS ownership details.	dnspython, python-whois

core/risk_engine.py	Applies rule-based weighting to scan flags to yield normalized security risk ratings.	Custom Python Logic
reports/	Formats scan results into downloadable PDF templates and raw CSV exports.	reportlab, pandas
db/	Manages SQLite connection pooling, schema migrations, and scan record CRUD operations.	sqlite3

6. Folder Structure

Plaintext

```
aegisx/
├── app.py                  # Main application entry point for Streamlit UI
├── requirements.txt        # Package dependencies
├── README.md               # Repository documentation
├── aegisx.log              # Standard logging output
├── config.py               # Global constants and default configurations
│
├── core/                   # Core scanning and logic modules
│   ├── __init__.py
│   ├── validator.py        # Input regex validation & IP resolver
│   ├── orchestrator.py     # Thread manager & module workflow controller
│   ├── port_scanner.py     # Async port scanner & banner grabber
│   ├── header_analyzer.py  # HTTP security header auditing engine
│   ├── ssl_inspector.py    # SSL/TLS certificate inspection logic
│   ├── recon.py            # DNS record & WHOIS lookup handlers
│   └── risk_engine.py      # Weighted risk score evaluation algorithm
│
├── db/                     # Persistence layer
│   ├── __init__.py
│   ├── database.py         # SQLite connection management
│   └── models.py           # Data models & SQL query constants
│
├── reports/                # Export builders
│   ├── __init__.py
│   ├── pdf_generator.py    # ReportLab PDF compilation module
│   └── csv_generator.py    # Pandas CSV dataset builder
│
├── ui/                     # Streamlit UI layouts & components
```

```
│   ├── __init__.py
│   ├── components.py          # Reusable metric cards, gauges, charts
│   └── styles.css             # Custom dashboard styling overrides
│
└── tests/                     # Automated unit and integration tests
    ├── test_validator.py
    ├── test_risk_engine.py
    └── test_port_scanner.py
```

7. Data Flow Diagram (DFD - Level 1)

```
Code snippet
graph TD
    User((User)) -- 1. Submit Target Domain/IP --> Validation[1.0 Validate & Sanitize Input]
    Validation -- Valid Target --> Orchestrator[2.0 Scan Orchestration Manager]
    Validation -- Invalid Target --> User

    subgraph Parallel Execution
        Orchestrator -- Target Params --> PortEngine[3.1 Port Scan & Banner Capture]
        Orchestrator -- Target Params --> WebEngine[3.2 HTTP Header & SSL Audit]
        Orchestrator -- Target Params --> ReconEngine[3.3 DNS & WHOIS Probe]
    end

    PortEngine -- Raw Open Ports --> Scorer[4.0 Heuristic Risk Scoring Engine]
    WebEngine -- Missing Headers & SSL Flags --> Scorer
    ReconEngine -- DNS / Reg Info --> Scorer

    Scorer -- Formatted Vulnerability Record --> Persistence[(5.0 SQLite Store)]
    Scorer -- Complete Result Object --> Visualization[6.0 UI Dashboard Visualizer]
    Scorer -- Complete Result Object --> Exporter[7.0 PDF / CSV Report Engine]

    Visualization -- Render Views --> User
    Exporter -- Output Files --> User
```

8. Sequence Diagram

This diagram demonstrates a complete scan pipeline execution from the front end to background modules and output generation.

```
Code snippet
sequenceDiagram
    autonumber
    actor User
    participant UI as Streamlit UI
    participant Val as Validator
    participant Orch as Scan Orchestrator
    participant Engines as Core Engines
    participant Risk as Risk Engine
```

```
    participant DB as SQLite DB
    participant Rep as Report Engine

    User->>UI: Enter Target Domain & Click "Start Scan"
    UI->>Val: validate_input(target_string)
    alt Invalid Input
        Val-->>UI: Return Regex Validation Error
        UI-->>User: Display Error Alert
    else Valid Input
        Val-->>UI: Return Clean Domain/IP Object
        UI->>Orch: execute_scan(clean_target)

        par Parallel Probing
            Orch->>Engines: run_port_scan(target)
            Orch->>Engines: inspect_ssl_certificate(target)
            Orch->>Engines: audit_http_headers(target)
            Orch->>Engines: resolve_dns_and_whois(target)
        end

        Engines-->>Orch: Return Raw Probe Data
        Orch->>Risk: compute_risk_score(raw_data)
        Risk-->>Orch: Return Calculated Risk Object (Score + Findings + Remediations)

        Orch->>DB: save_scan_record(risk_object)
        DB-->>Orch: Confirm Transaction Success

        Orch->>Rep: build_reports(risk_object)
        Rep-->>Orch: Return PDF Bytes & CSV Strings

        Orch-->>UI: Return Final Scan State
        UI-->>User: Render Metrics, Charts, and Download Buttons
    end
```

9. Class Diagram

```
Code snippet
classDiagram
    class ScanOrchestrator {
        +TargetValidator validator
        +PortScanner port_scanner
        +HeaderAnalyzer header_analyzer
        +SSLInspector ssl_inspector
        +ReconEngine recon_engine
        +RiskEngine risk_engine
        +execute_scan(target: str) ScanResult
    }

    class TargetValidator {
```

```
    +validate_domain(target: str) bool
    +validate_ip(target: str) bool
    +sanitize(target: str) str
}

class PortScanner {
    +int timeout
    +list default_ports
    +async scan_ports(target: str) list~PortFinding~
    +grab_banner(ip: str, port: int) str
}

class HeaderAnalyzer {
    +audit_headers(url: str) list~HeaderFinding~
}

class SSLInspector {
    +inspect_certificate(domain: str, port: int) SSLResult
}

class RiskEngine {
    +float max_score
    +calculate_score(findings: list) ScoreReport
}

class ScanResult {
    +str scan_id
    +str target
    +float risk_score
    +list findings
    +datetime timestamp
}

ScanOrchestrator --> TargetValidator
ScanOrchestrator --> PortScanner
ScanOrchestrator --> HeaderAnalyzer
ScanOrchestrator --> SSLInspector
ScanOrchestrator --> RiskEngine
ScanOrchestrator ..> ScanResult
```

10. Activity Diagram

Code snippet

```
stateDiagram-v2
    [*] --> Idle: Application Loaded
    Idle --> ValidatingInput: User Submits Target Domain/IP

    state ValidatingInput {
```

```
    [*] --> CheckRegex
    CheckRegex --> InputValid: Pattern Match
    CheckRegex --> InputInvalid: Malicious Characters / Bad Domain
  }

  InputInvalid --> Idle: Display Warning Message
  InputValid --> InitializingScan: Instantiate Scan Context

  state InitializingScan {
    [*] --> ResolveIP
    ResolveIP --> LaunchThreads: Asynchronous Socket Initialization
  }

  state LaunchThreads {
    [*] --> PortScanState: Scan Ports & Grab Banners
    [*] --> HeaderAuditState: Check HTTP/HTTPS Headers
    [*] --> SSLCheckState: Query Port 443 Cert Specs
    [*] --> ReconState: Fetch DNS & WHOIS Records
  }

  PortScanState --> AggregatingResults
  HeaderAuditState --> AggregatingResults
  SSLCheckState --> AggregatingResults
  ReconState --> AggregatingResults

  AggregatingResults --> CalculatingRiskScore: Calculate Heuristics Weights
  CalculatingRiskScore --> SavingToDatabase: Persist Scan Metadata
  SavingToDatabase --> GeneratingReports: Compile PDF/CSV Artifacts
  GeneratingReports --> DisplayingResults: Update Dashboard UI
  DisplayingResults --> Idle: Ready for Next Scan
```

11. Use Case Diagram

```
Code snippet
graph TD
  Developer([DevSecOps / Engineer])
  Analyst([Security Analyst])
  Recruiter([Technical Assessor])

  subgraph AegisX Assessment Platform
    UC1(Execute Combined Security Audit)
    UC2(View Attack Surface Findings)
    UC3(Inspect SSL/TLS Configuration)
    UC4(Review Contextual Remediations)
    UC5(Export Executive PDF Report)
    UC6(Export Raw Findings to CSV)
    UC7(View Saved Historical Scans)
  end
```

```
Developer --> UC1
Developer --> UC2
Developer --> UC4

Analyst --> UC1
Analyst --> UC3
Analyst --> UC5
Analyst --> UC6

Recruiter --> UC1
Recruiter --> UC5
Recruiter --> UC7
```

12. Technology Stack

- **Programming Language:** Python 3.10+
- **User Interface:** Streamlit Framework
- **Data Visualization:** Plotly Express / Altair
- **Asynchronous Execution:** Python asyncio, concurrent.futures
- **Networking & Protocols:** socket, ssl, requests, dnspython, python-whois
- **PDF Compile Engine:** ReportLab
- **Data Formatting:** Pandas
- **Persistence:** SQLite3

13. Database Design

AegisX uses a relational schema designed in SQLite to support scan persistence and historical trend queries.

```
Code snippet
erDiagram
    TARGETS ||--o{ SCANS : has
    SCANS ||--|{ FINDINGS : contains
    SCANS ||--o| REPORTS : generates

    TARGETS {
        string target_id PK
        string host_identifier
        string target_type
        datetime created_at
    }

    SCANS {
        string scan_id PK
        string target_id FK
        datetime execution_time
        float overall_risk_score
```

```
    int ports_scanned
    int open_ports_count
    string status
}

FINDINGS {
    int finding_id PK
    string scan_id FK
    string module_category
    string severity_level
    string finding_title
    string description
    string remediation_advice
}

REPORTS {
    string report_id PK
    string scan_id FK
    string pdf_storage_path
    string csv_storage_path
    datetime generated_at
}
```

14. API Design (Future-Ready REST Engine)

To support transition toward a headless cloud architecture, AegisX defines OpenAPI-compliant REST endpoints:

POST /api/v1/scans

Initiates a new multi-module security scan.

- **Request Payload:**
- JSON

```
{
  "target": "example.com",
  "scan_profile": "standard",
  "ports": [80, 443, 22, 21, 8080]
}
```

-
-
- **Response Payload (202 Accepted):**
- JSON

```
{
  "scan_id": "aegis-exec-89df23a1",
  "status": "QUEUED",
  "estimated_duration_seconds": 15
```

}

-
-

GET /api/v1/scans/{scan_id}

Retrieves complete scan result, risk breakdown, and individual findings.

- **Response Payload (200 OK):**
- JSON

```
{
  "scan_id": "aegis-exec-89df23a1",
  "target": "example.com",
  "risk_score": 4.5,
  "severity": "MEDIUM",
  "summary": {
    "open_ports": [80, 443],
    "missing_headers": ["Content-Security-Policy", "Strict-Transport-Security"]
  }
}
```

-
-

GET /api/v1/scans/{scan_id}/report?format=pdf

Downloads the generated ReportLab PDF audit document.

15. Risk Scoring Workflow

Risk evaluation translates qualitative findings into a normalized quantitative metric ($0.0 - 10.0$).

Code snippet

```
flowchart TD
    Start([Collect Scanned Findings]) --> BaseScore[Initialize Base Risk Score = 0.0]

    BaseScore --> PortCheck{Check Open Ports}
    PortCheck -- Sensitive Port Open e.g., 21, 23, 3389 --> AddPortRisk[Score += 2.5 per port]
    PortCheck -- Common Port Open e.g., 80, 443 --> AddLowPortRisk[Score += 0.2 per port]

    AddPortRisk --> HeaderCheck{Audit Web Headers}
    AddLowPortRisk --> HeaderCheck

    HeaderCheck -- Missing Critical Header e.g., CSP, HSTS --> AddHeaderRisk[Score += 1.0 per header]
    HeaderCheck -- Missing Secondary Header --> AddLowHeaderRisk[Score += 0.5 per header]
```

```
    AddHeaderRisk --> SSLCheck{Inspect SSL/TLS Cert}
    AddLowHeaderRisk --> SSLCheck

    SSLCheck -- Expired or Self-Signed --> AddSSLHigh[Score += 3.0]
    SSLCheck -- Expiring within 30 days --> AddSSLMed[Score += 1.5]

    AddSSLHigh --> CapCheck{Calculate Total Sum}
    AddSSLMed --> CapCheck

    CapCheck --> CapScore[Risk Score = MIN 10.0, Sum]
    CapScore --> MapSeverity{Evaluate Severity Tier}

    MapSeverity -- Score >= 7.0 --> High[HIGH RISK]
    MapSeverity -- 4.0 <= Score < 7.0 --> Med[MEDIUM RISK]
    MapSeverity -- Score < 4.0 --> Low[LOW RISK]

    High --> End([Return Final Risk Model])
    Med --> End
    Low --> End
```

16. Security Analysis Workflow

```
Code snippet
sequenceDiagram
    participant Worker as Core Security Engine
    participant Target as Target Endpoint

    Note over Worker,Target: Step 1: TCP Handshake Probing
    Worker->>Target: Asynchronous TCP Connect Request (Port 80/443/etc.)
    Target-->>Worker: TCP SYN-ACK / Banner Output

    Note over Worker,Target: Step 2: Protocol Handshake Verification
    Worker->>Target: TLS ClientHello Connection Request
    Target-->>Worker: TLS ServerHello + X.509 Certificate Chain

    Note over Worker,Target: Step 3: Web Application Response Auditing
    Worker->>Target: HTTP GET / HEAD Request
    Target-->>Worker: HTTP Response Headers (Inspect X-Frame, HSTS, CSP)

    Note over Worker,Target: Step 4: Passive Intelligence
    Worker->>Target: DNS Query (UDP 53) & WHOIS Port 43 Probes
    Target-->>Worker: Resolved Records & Registrar Objects
```

17. Report Generation Workflow

```
Code snippet
flowchart LR
    ScanData[(Scan Results Object)] --> PDFEngine[ReportLab PDF Engine]
```

```
    ScanData --> CSVEngine[Pandas Dataframe Engine]

    subgraph PDF Processing Pipeline
        PDFEngine --> ApplyStyles[Apply Typography & Palettes]
        ApplyStyles --> DrawHeaders[Draw Cover & Metadata]
        DrawHeaders --> RenderScore[Render Risk Score & Meter]
        RenderScore --> BuildTable[Compile Findings & Remediations Table]
        BuildTable --> OutputPDF[Render Binary PDF Stream]
    end

    subgraph CSV Processing Pipeline
        CSVEngine --> FlattenData[Flatten Scan Array to 2D Format]
        FlattenData --> ExportCSV[Generate Raw CSV Data File]
    end

    OutputPDF --> UIStream[Streamlit File Download Triggers]
    ExportCSV --> UIStream
```

18. Dashboard Architecture

The dashboard is structured around custom Streamlit layouts and Plotly charts to maintain smooth render updates during scan tasks.

```
Code snippet
graph TD
    A[app.py Main Dashboard Entry] --> B[Sidebar Controls & Validation]
    A --> C[Main Content Layout Container]

    C --> D[Header & Metadata Panel]
    C --> E[Metric Gauges & Plotly Score Visualizer]
    C --> F[Tabbed Scan Findings Display]

    subgraph Result Tabs
        F --> Tab1[Network Ports & Services Table]
        F --> Tab2[HTTP Headers Audit Matrix]
        F --> Tab3[SSL/TLS Security Analysis]
        F --> Tab4[DNS & WHOIS Intelligence]
        F --> Tab5[Actionable Remediation Checklist]
    end

    C --> G[Download Action Bar - PDF & CSV Buttons]
```

19. Core Design Principles

- **Single Responsibility Principle (SRP):** Each scanner component performs one specific protocol audit task and returns structured data types without managing UI state.

- **Loose Coupling:** The scan orchestrator relies on dependency injection abstractions, allowing scanner modules to be added, modified, or swapped independently.
- **Fail-Safe Defensive Execution:** Socket operations feature explicit timeout boundaries and fallbacks to handle silent or non-responsive firewalls safely.
- **Zero Command-Line Dependency:** All networking tasks use native Python standard and user-space socket protocols, eliminating external dependencies on OS-level CLI tools (e.g., nmap binaries).

20. Scalability Considerations

- **Concurrent Execution:** Port scanning tasks leverage asyncio loop pools, processing dozens of connection attempts in parallel.
- **Resource Optimization:** Sockets are closed immediately after banner reading to avoid leaving idle socket descriptors open.
- **Database Optimization:** Database write operations use parameterized connection wrappers with indices created on scan_id and target_id attributes.

21. Future Expansion Architecture

Code snippet
graph TD
CurrentCore[AegisX Core Orchestrator] --> AI_Module[AI Explanation Layer - OpenAI/Gemini]
CurrentCore --> CVE_Module[NIST NVD API Lookup Engine]
CurrentCore --> Threat_Module[Shodan / VirusTotal / AbuseIPDB APIs]

AI_Module --> ContextRemediation[Context-Aware Remediation Engine]
CVE_Module --> VulnerabilityMapping[Automated CVE Identifier Mapping]
Threat_Module --> ThreatScore[Reputation Score Engine]

ContextRemediation --> UnifiedReport[AI Executive Report]
VulnerabilityMapping --> UnifiedReport
ThreatScore --> UnifiedReport

- **AI Security Context Layer:** Direct integration with modern Large Language Models to contextualize identified missing headers and open ports into step-by-step remediation commands tailored to specific web server platforms (e.g., NGINX, Apache).
- **Automated CVE Mapping:** Integration with NIST National Vulnerability Database (NVD) endpoints to correlate extracted service banners with known CVE listings.
- **Threat Intelligence Feeds:** Querying external IP threat APIs to incorporate reputation metrics into the risk scoring logic.